

Conditionals and Control Flow

4

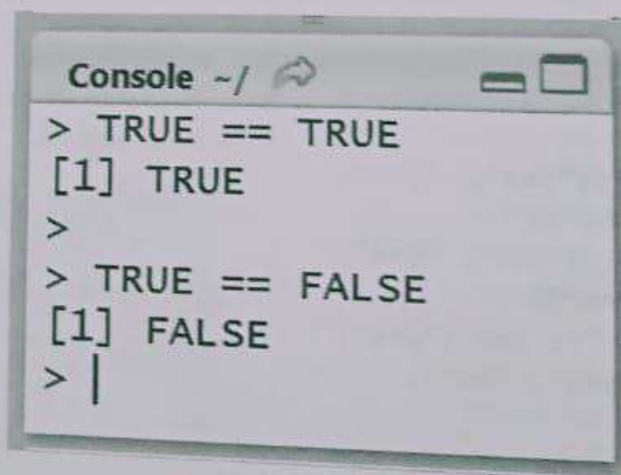
Learning Objectives

A systematic study of this chapter will help the reader understand the following concepts:

- Relational operators
- Logical operators—**and**, **or**, **not**
- Operators and vectors
- Conditional statements

4.1 RELATIONAL OPERATORS

Relational operators are comparators that help us understand how one object relates to another. For example, to check whether two objects are equal, the double equal operator can be used to check if the logical value **TRUE** equals the logical value **TRUE** (**TRUE == TRUE**) and the result of this query is a logical value, in this case **TRUE**; on the contrary **TRUE == FALSE** will return **FALSE**, as shown in Fig. 4.1.



```
Console ~/     
> TRUE == TRUE  
[1] TRUE  
>  
> TRUE == FALSE  
[1] FALSE  
> |
```

Fig. 4.1 Equality check

Logical variables can also be used to check the quality of other types, which compares strings and numbers. The opposite of the equality operator is the inequality operator, written as an exclamation mark followed by an equal sign (**!=**). For example, the output of **"hello != goodbye"** will be **TRUE**. Equality operator can also be used for numeric, logical, and other objects. Every time the result of the equality operator is the opposite as that of the inequality operator, as shown in Fig. 4.2.

```

Console ~/
> "hello"=="GoodBye"
[1] FALSE
>
> 3==2
[1] FALSE
>
> TRUE!=TRUE
[1] FALSE
>
> TRUE!=FALSE
[1] TRUE
>
> "hello"!="GoodBye"
[1] TRUE
>
> 3!=2
[1] TRUE
> |

```

Fig. 4.2 Comparing strings, numbers, and logical values

These are the steps needed to be done to check if an object is lesser than or greater than another object. In the case of numerical values $3 < 5$ will evaluate to TRUE and $3 > 5$ will evaluate to FALSE. For character strings and logical values, it uses the alphabetic order to sort character strings while evaluating "Hello" > "Goodbye", which results in TRUE. Since H comes after G in the alphabet, Hello is considered greater than Goodbye. For logical values is $\text{TRUE} < \text{FALSE}$ evaluates to FALSE, since TRUE corresponds to 1 and FALSE corresponds to 0; hence the result FALSE, as shown in Fig. 4.3.

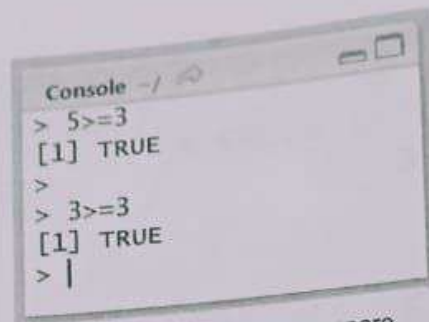
To check if one object is greater than or equal to/less than or equal to another object, the less than or equal to/greater than or equal to operator can be used. For example $5 \geq 3$ and $3 \geq 5$ will evaluate to TRUE, as shown in Fig. 4.4.

```

Console ~/
> 3<5
[1] TRUE
>
> 3>5
[1] FALSE
>
> "Hello">"Goodbye"
[1] TRUE
>
> TRUE < FALSE
[1] FALSE
> |

```

Fig. 4.3 Use of relational operators

Fig. 4.4 Using `>=` to compare

4.2 RELATIONAL OPERATORS AND VECTORS

Consider a scenario for recording the number of views of a LinkedIn profile per day using the vector *linkedin*, shown in Fig. 4.5.

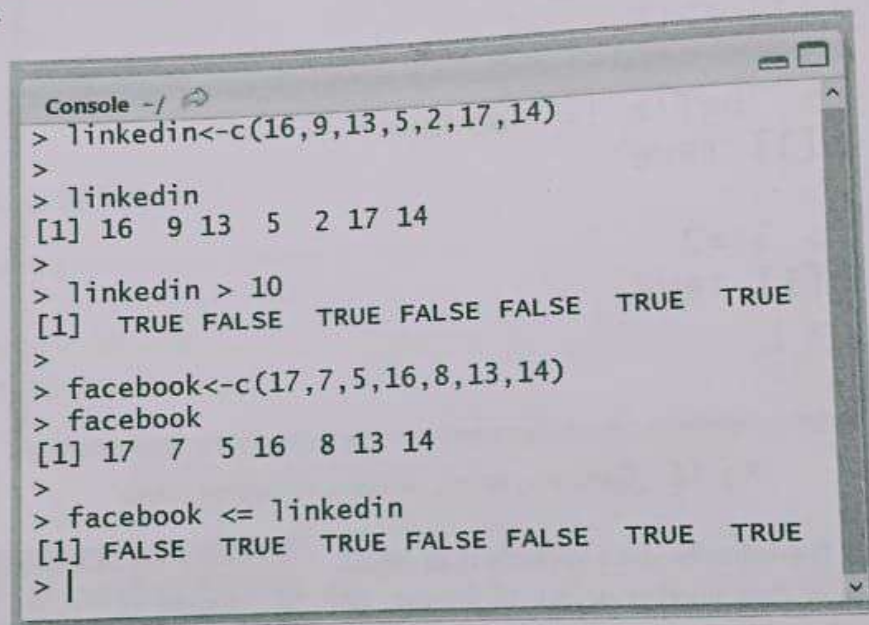


Fig. 4.5 Relational operators and vectors

For example, to find out on which days the number of views exceeded 10, use the greater than (`>`) operator. For the 1st, 3rd, 6th and 7th elements in the vector, the number of views is greater than 10, and hence for these elements the result will be TRUE.

Vectors can also be compared with other vectors. A record of the number of facebook user profiles for the previous week is available, which can be saved in another vector called *facebook*. Evaluation of the number of Facebook views less than or equal to the number of LinkedIn views is illustrated in Fig. 4.5. In this case the comparison is done for every element of the vector, one by one. For example, on the third day, the number of Facebook views is 5 and the number of LinkedIn views is 13, and hence the comparison evaluates to TRUE as 5 is smaller than or equal to 13.

4.3 LOGICAL OPERATORS

To change the combined results of comparisons, R uses the logical operators—AND (`&`), OR (`|`) and NOT (`!`).

4.3.1 AND Operator

The AND operator takes two logical values and returns TRUE only if both these logical values are each TRUE; this means that TRUE and TRUE evaluates to TRUE, but FALSE and TRUE, TRUE and FALSE, and FALSE and FALSE, all evaluate to FALSE, as shown in Fig. 4.6.

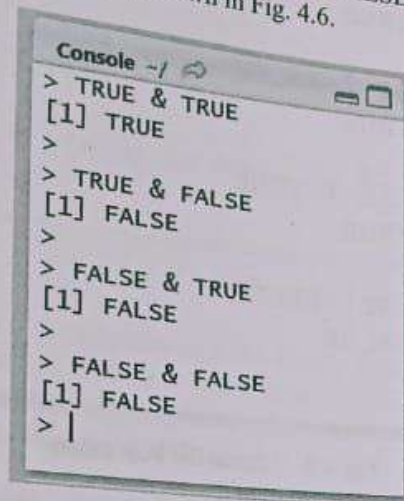


Fig. 4.6 Logical AND truth values

Instead of using logical values, the results of comparisons can be used. For example, take a variable $X = 12$. To check if the variable is greater than 5 but less than 15, $X > 5$ and $X < 15$ can be used. The first part will evaluate to TRUE and the second part will also evaluate to TRUE and hence the result of this expression is TRUE, since 12 lies between 5 and 15. If $X = 17$, the expression $X > 5$ and $X < 15$ would simplify to TRUE and FALSE, which results in this expression being FALSE, as shown in Fig. 4.7.

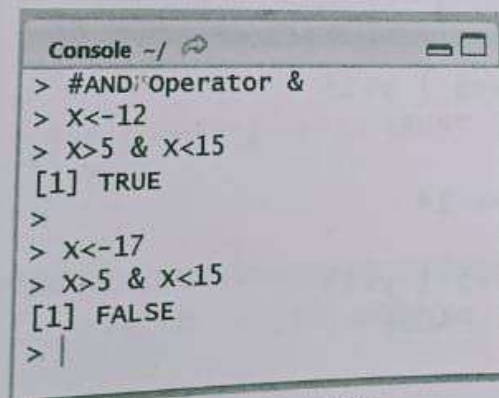
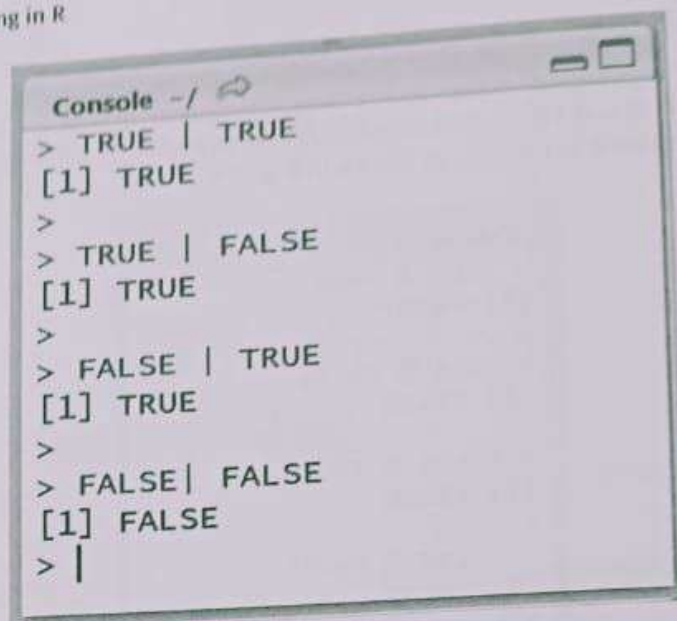


Fig. 4.7 Logical AND

4.3.2 OR Operator

The OR operator works similar to the AND operator with a difference that at least one of the logical values should be TRUE for the entire operation to evaluate to TRUE. This means that not only TRUE or TRUE should be TRUE for the entire operation to evaluate to TRUE, but also TRUE or FALSE and FALSE or TRUE evaluate to TRUE. When both the logical evaluates to TRUE, but also TRUE or FALSE, the expression results in FALSE as shown in Fig. 4.8.

Remember that this operation is not an exclusive OR (XOR) operation, and so TRUE or TRUE equals TRUE as well. For example, assume variable $y = 5$; to see if this variable is < 5 or > 15 , $y < 5 \mid y > 15$ can



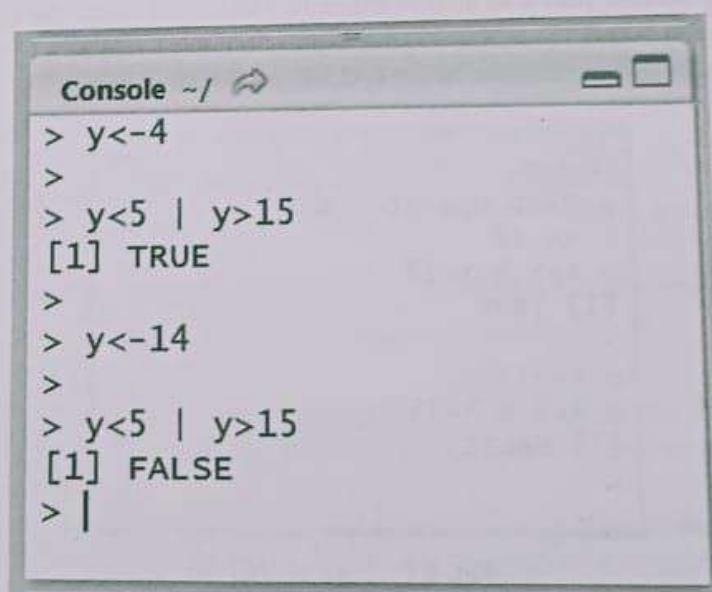
```

Console ~/
> TRUE | TRUE
[1] TRUE
>
> TRUE | FALSE
[1] TRUE
>
> FALSE | TRUE
[1] TRUE
>
> FALSE | FALSE
[1] FALSE
> |

```

Fig. 4.8 Logical OR truth values

be used. R first carries out the individual comparisons resulting in TRUE or FALSE, which in turn results in TRUE. If $y = 14$, the expression $y < 5 \mid y > 15$ now evaluates to FALSE or FALSE, neither one of the comparisons is TRUE, and so the result is FALSE as shown in Fig. 4.9.



```

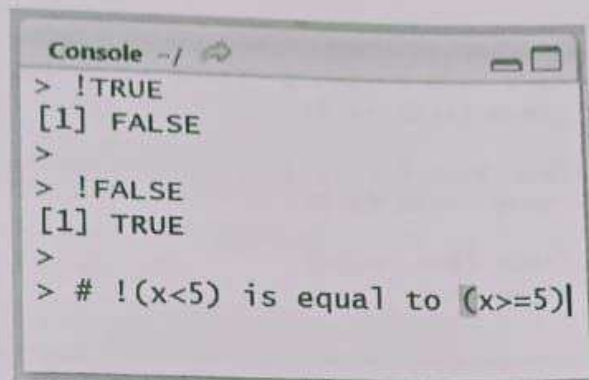
Console ~/
> y<-4
>
> y<5 | y>15
[1] TRUE
>
> y<-14
>
> y<5 | y>15
[1] FALSE
> |

```

Fig. 4.9 Logical OR

4.3.3 NOT Operator

The NOT operator represented by ! simply negates the given logical value. In this regard !TRUE evaluates to FALSE and !FALSE evaluates to TRUE. Similar to the OR and AND operators, the NOT operator can be used in combination with other logical operators. This is not always necessary however, because this line of code $!(x < 5)$ is exactly the same as $x \geq 5$, as shown in Fig. 4.10.



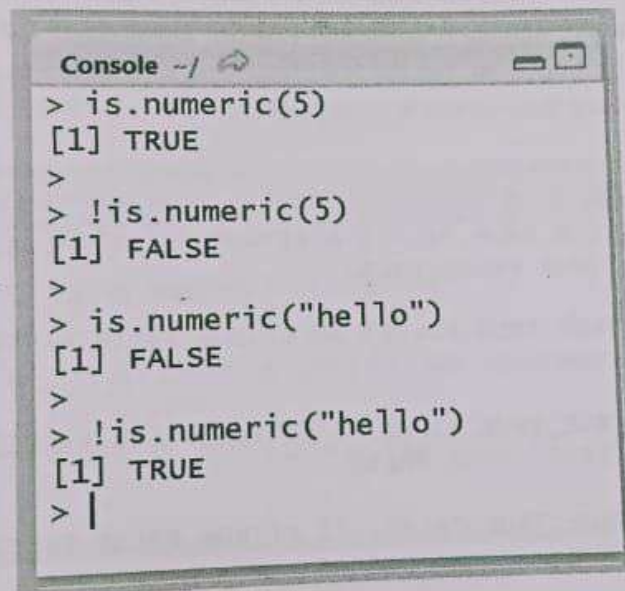
```

Console ~/
> !TRUE
[1] FALSE
>
> !FALSE
[1] TRUE
>
> # !(x<5) is equal to (x>=5)

```

Fig. 4.10 NOT operator

There are cases in R where the NOT operator is really handy. Take for example the built-in R function *is.numeric* that checks if an R object is a numeric value. *is.numeric(5)* evaluates to TRUE as 5 is numeric. To negate this result, we use the NOT operator. If we type *is.numeric(hello)*, it will be evaluated to FALSE; negating this result gives TRUE, as shown in Fig. 4.11.



```

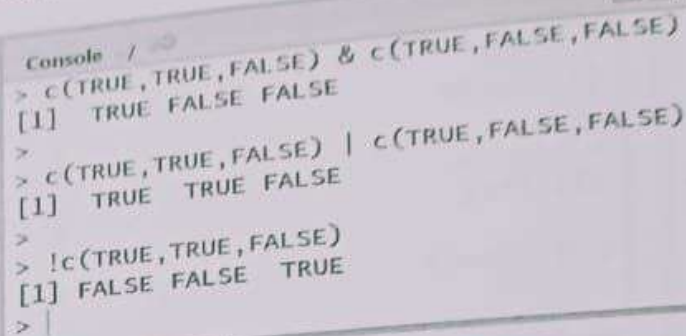
Console ~/
> is.numeric(5)
[1] TRUE
>
> !is.numeric(5)
[1] FALSE
>
> is.numeric("hello")
[1] FALSE
>
> !is.numeric("hello")
[1] TRUE
> |

```

Fig. 4.11 Using NOT operator

4.4 LOGICAL OPERATORS AND VECTORS

The working of logical operators with vectors and matrices is similar to that of the relational operators; the operations are performed element-wise. This is shown in Fig. 4.12 where two vectors with the elements TRUE, TRUE, FALSE and TRUE, FALSE, FALSE are taken. AND operation on these vectors results in a vector with the elements TRUE, FALSE, and FALSE. The first elements in both factors are TRUE, so the first element of the resulting vector contains TRUE; similarly the second elements are TRUE and FALSE, which result in FALSE; and the third elements are FALSE and FALSE giving FALSE. Similarly the OR operation on these vectors gives the following result: TRUE or TRUE results in TRUE; TRUE or FALSE also gives TRUE; and FALSE or FALSE is FALSE. The NOT operator also works on every element of the vector, and every TRUE is converted to FALSE and FALSE is converted to TRUE.



```

Console /
> c(TRUE, TRUE, FALSE) & c(TRUE, FALSE, FALSE)
[1] TRUE FALSE FALSE
>
> c(TRUE, TRUE, FALSE) | c(TRUE, FALSE, FALSE)
[1] TRUE TRUE FALSE
>
> !c(TRUE, TRUE, FALSE)
[1] FALSE FALSE TRUE
>

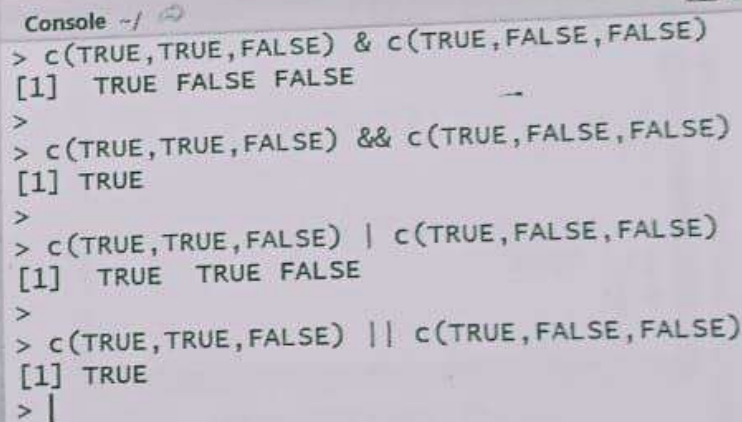
```

Fig. 4.12 Logical operators and vectors

4.5 & VS &&, | VS ||

There are two versions of the AND and OR operators, namely the single operator and the double operator versions. R can use either of these, and the result varies with the version used.

As shown in Fig. 4.13, again using the same vectors used in Fig. 4.12 as inputs, the first expression evaluates to a vector containing TRUE, FALSE, and FALSE. However if && is used, the result will be TRUE, since the && operator only examines the first element of each factor—in this case the first elements are TRUE and TRUE and hence the expression returns TRUE.



```

Console ~/
> c(TRUE, TRUE, FALSE) & c(TRUE, FALSE, FALSE)
[1] TRUE FALSE FALSE
>
> c(TRUE, TRUE, FALSE) && c(TRUE, FALSE, FALSE)
[1] TRUE
>
> c(TRUE, TRUE, FALSE) | c(TRUE, FALSE, FALSE)
[1] TRUE TRUE FALSE
>
> c(TRUE, TRUE, FALSE) || c(TRUE, FALSE, FALSE)
[1] TRUE
>

```

Fig. 4.13 Differentiating & and &&, | and ||

A similar scenario can happen with the OR operator—the single operator version returns an entire vector and double operator version returns only the OR result of the first element of each vector.

4.6 CONDITIONAL STATEMENTS

If-else

The syntax of an if statement contains a condition; if the condition evaluates to TRUE, the R code associated with the if statement is executed.

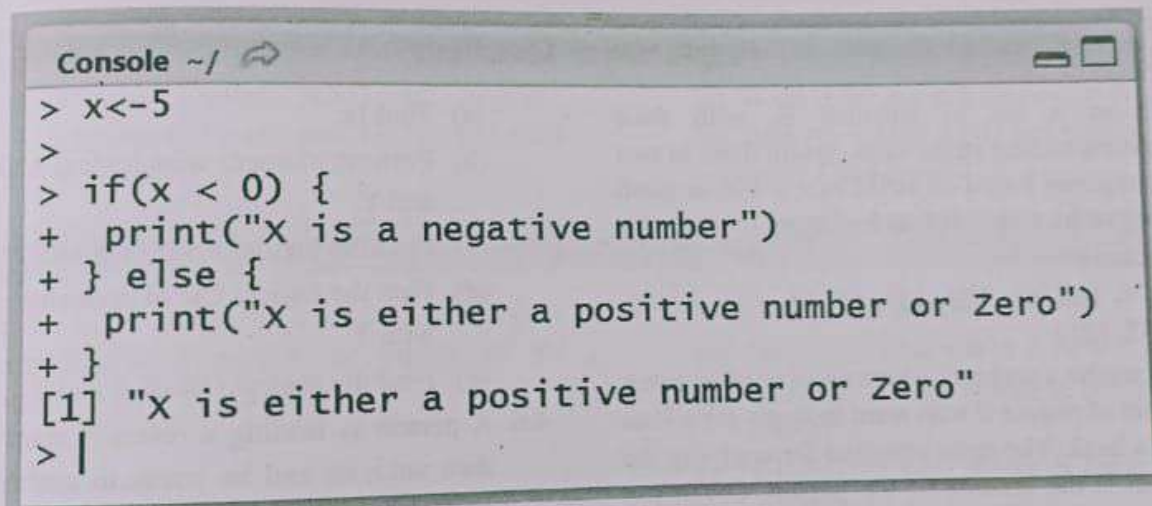
The condition appears inside parentheses, whereas the R code that has to be executed if the condition is true follows in curly brackets.

```
if(condition) {
    expression
}
```

In the case of an if-else statement, the code associated with the else statement gets executed whenever the condition of the if statement is not satisfied.

```
if(condition) {
    expression 1
}
else {
    expression 2
}
```

Figure 4.14 shows an if condition that checks if X is less than zero. It prints "X is either a positive number or zero" whenever the condition is not met. For the value $X = -3$, the output will be "X is a negative number" because the if condition is TRUE. On changing X to 5, the condition is not satisfied and so R prints the expression following the else statement.



```
Console ~/
> x <- -5
>
> if(x < 0) {
+   print("X is a negative number")
+ } else {
+   print("X is either a positive number or zero")
+ }
[1] "X is either a positive number or zero"
> |
```

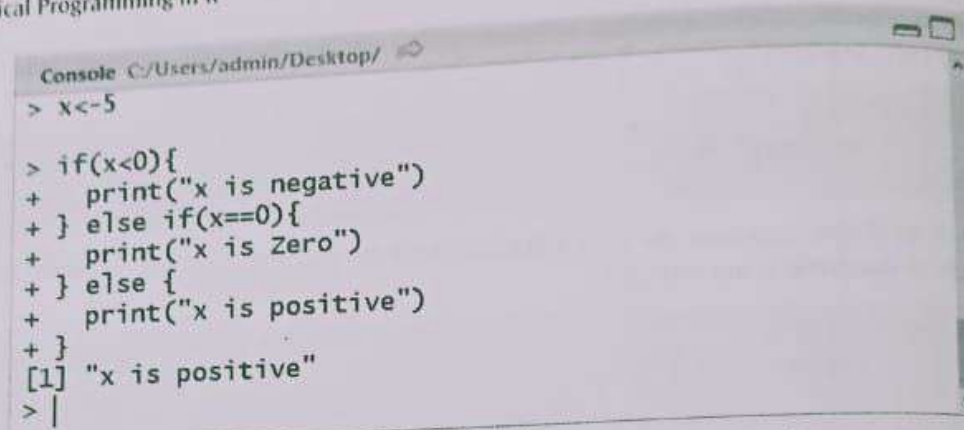
Fig. 4.14 Conditional statements

If-else if-else

There are also cases where programs are customized even further, for instance, in the example shown in Fig. 4.14, to print if X equals exactly zero. This is done using the elseif statement in R.

```
if(condition1) {
    expression 1
} else if(condition2) {
    expression 2
} else {
    expression 3
}
```

The else if statement comes in between the if and else statements. This is illustrated clearly in Fig. 4.15.



```

> x <- -5

> if(x < 0){
+   print("x is negative")
+ } else if(x == 0){
+   print("x is Zero")
+ } else {
+   print("x is positive")
+ }
[1] "x is positive"
> |

```

Fig. 4.15 Using logical operators in conditional statements

In this case, when $X = -3$, the condition for the if statement evaluates to TRUE and hence "X is a negative number" gets printed and R ignores the rest of the statements. If $X = 0$, R will first check the if condition, see that it is FALSE, and then moves over to the else if condition. The condition $x == 0$ evaluates to TRUE and so "X is zero" gets printed and R ignores the else statement. Finally when $X = 5$, the if condition evaluates to FALSE and so does the else if condition, and so R executes the else statement printing "X is positive number" as shown in the figure.

Programming Questions

- 4.1 Given a list of batsmen B, with their corresponding strike rates, group them in two categories based on strike rate > 100 as good and strike rate < 100 as bad using Ifelse().
 $B \leftarrow \text{list}(100, 123, 153, 65, 75, 123, 43, 45, 132, 121)$
- 4.2 Consider a problem where users need to group a set of people P who want to apply for a loan at a bank. The main criterion for applying the loan is the income of the person. Given $I = (10000, 20000, 30000, 40000, 12313, 13141, 43212, 23231)$, which corresponds to the respective incomes of the people in P. Group the people into low risk customer or high risk customer, based on whether the customer's income is above 30000 (low risk) or not (high risk).
- 4.3 Write an R program to check the eligibility for voting using if-else condition.
- 4.4 Write an R program to print student grades using if-else.
- 4.5 Assign the values 'TRUE', 'FALSE', 'FALSE', 'TRUE' to logical vector 'X' and 'FALSE', 'TRUE', 'FALSE', 'TRUE' to logical vector 'Y'.
 - (a) Find !x.
 - (b) Perform element wise logical AND on X and Y.
 - (c) Find the logical AND of X and Y.
 - (d) Find the logical OR of each element of X with Y.
 - (e) Find the logical OR of X and Y.
- 4.6 A person is leading a research team of five data analysts and he wants to evaluate their performance. As a first step, assign a *project_complete* vector and *work_time* vector with 5 entries, one for each analyst. To the *project_complete* vector, assign 0 if the project is not completed or 1 if the project is completed. Given the following data:
 - Analyst 1 has completed the given project, worked for 4 hours.
 - Analyst 2 has not completed the project worked for 1 hour.
 - Analyst 3 has not completed the project worked for 0 hours.
 - Analyst 4 has completed the given project, worked for 5 hour.